

33-AI-ERROR-HANDBOOK.md

Version : V1.0

Statut : ACTIVE

Catégorie : AI Quality Framework

Projet : Vitala

1. Objectif

AI-ERROR-HANDBOOK.md définit les règles officielles de gestion des erreurs dans Vitala.

Il établit :

- comment détecter les erreurs ;
 - comment les classer ;
 - comment les traiter ;
 - comment les documenter ;
 - comment éviter leur répétition.
-

2. Principe fondamental

Une erreur n'est pas seulement un problème technique.

C'est un événement qui doit être :

- compris ;
 - contrôlé ;
 - expliqué ;
 - corrigé.
-

3. Principes obligatoires

Toute erreur doit respecter :

Transparence

Le système doit savoir qu'une erreur existe.

Sécurité

Ne jamais exposer d'informations sensibles.

Récupération

Lorsque possible, le système doit continuer à fonctionner.

Traçabilité

Les erreurs importantes doivent être enregistrées.

4. Classification officielle des erreurs

Les erreurs Vitala sont classées :

ERROR LEVEL 0 — Information

Situation normale nécessitant une information.

Exemples :

- synchronisation terminée ;
 - action réussie.
-

ERROR LEVEL 1 — Warning

Problème mineur sans impact critique.

Exemples :

- réseau lent ;
- fonctionnalité temporairement limitée.

Action :

Informez l'utilisateur.

ERROR LEVEL 2 — Recoverable Error

Erreur récupérable automatiquement.

Exemples :

- perte réseau ;
- synchronisation temporairement impossible.

Action :

Réessayer ou proposer une alternative.

ERROR LEVEL 3 — Functional Error

Erreur empêchant une action utilisateur.

Exemples :

- données invalides ;
- permission insuffisante.

Action :

Afficher une explication claire.

ERROR LEVEL 4 — System Error

Erreur technique importante.

Exemples :

- service indisponible ;
- problème serveur.

Action :

Journaliser et surveiller.

ERROR LEVEL 5 — Critical Error

Erreur mettant en danger :

- sécurité ;

- données ;
- disponibilité.

Action :

Blocage immédiat + alerte.

5. Format standard d'une erreur

Toute erreur interne doit suivre :

``text id="e7m2qa" Error ID :

Type :

Niveau :

Module :

Message technique :

Message utilisateur :

Cause probable :

Action recommandée :

```
---
```

```
# 6. Messages utilisateur
```

```
Les messages doivent être :
```

- ```
- simples ;
- compréhensibles ;
- utiles.
```

```
Mauvais :
```

```
"NullPointerException database service."
```

```
Bon :
```

```
"Impossible de charger vos données actuellement. Vérifiez votre connexion
puis réessayez."
```

```

```

## # 7. Erreurs Frontend

L'interface doit gérer :

### ## Chargement

Afficher un état clair.

---

### ## Erreur

Afficher une solution.

---

### ## Vide

Expliquer pourquoi aucune donnée n'existe.

---

### ## Permission

Indiquer pourquoi l'accès est limité.

---

## # 8. Erreurs Backend

Le backend doit :

- capturer les exceptions ;
- retourner des erreurs structurées ;
- protéger les détails internes.

---

## # 9. Erreurs API

Format recommandé :

```
```json
{
  "error": {
    "code": "AUTH_REQUIRED",
    "message": "Authentification nécessaire",
    "requestId": "xxx"
  }
}
```

10. Erreurs Database

Les erreurs Database doivent gérer :

- contraintes violées ;
- connexion impossible ;
- migration incorrecte ;
- permissions RLS.

Ne jamais exposer directement les erreurs PostgreSQL à l'utilisateur.

11. Erreurs Offline First

Les modules offline doivent gérer :

Perte réseau

Continuer localement.

Synchronisation échouée

Conserver les données locales.

Conflit

Détecter et résoudre.

Données corrompues

Restaurer une version valide.

12. Erreurs Synchronisation

Une erreur Sync doit contenir :

``text id="r8q2mv" Opération concernée :

Date :

Utilisateur :

Tentatives :

Dernière erreur :

Solution :

13. Erreurs Intelligence Artificielle

Les fonctions IA doivent gérer :

- modèle indisponible ;
- réponse invalide ;
- dépassement limite ;
- coût inattendu.

14. Logs erreurs

Les logs doivent contenir :

- date ;
- module ;
- niveau ;
- contexte technique.

Interdit :

- mots de passe ;
- tokens ;
- données privées inutiles.

15. Gestion automatique des erreurs

Lorsque possible :

Le système peut :

- réessayer ;
- mettre en attente ;
- utiliser un fallback ;
- informer l'utilisateur.

16. Analyse après incident

Après une erreur importante :

Documenter :

```text id="m5v8zt"

Description :

Cause :

Impact :

Correction :

Prévention :

---

## 17. Tests d'erreurs obligatoires

Une fonctionnalité doit tester :

☐ Cas normal

☐ Données invalides

☐ Absence permission

☐ Réseau indisponible

☐ Service externe indisponible

---

## 18. Interdictions

Une IA ne doit jamais :

✗ cacher une erreur.

✗ ignorer une exception.

✗ afficher des détails techniques sensibles.

✗ supprimer un contrôle pour éviter une erreur.

---

## 19. Checklist erreur

Avant validation :

☐ Erreurs prévues

☐ Messages utilisateur définis

☐ Logs configurés

☐ Récupération prévue

☐ Tests réalisés

---

## 20. Règle finale

Un système professionnel n'est pas celui qui ne rencontre jamais d'erreur.

C'est celui qui sait :

- détecter ;
  - expliquer ;
  - récupérer ;
  - apprendre.
- 

## Historique

| Version | Modification                 |
|---------|------------------------------|
| V1.0    | Création du guide erreurs IA |

---

**Fin de AI/33-AI-ERROR-HANDBOOK.md**